

递归

1. 计算数字根

小明想吃糖果。小明的爸爸要求小明必须在规定时间内计算出任意整数的数字根，才能吃到糖果。一个整数的数字根（digital root）是指不断重复地将各位数字相加，直到结果只有一位数字为止。例如，1729 的数字根的计算步骤如下：

步骤 1: $1+7+2+9 \rightarrow 19$

步骤 2: $1+9 \rightarrow 10$

步骤 3: $10 \rightarrow 1$

因此 1729 的数字根是 1。

请你编写一个程序帮助小明计算任意正整数的数字根。

要求：首先编写递归函数 `int digitalRoot(int n)` 该函数返回参数 `n` 的数字根，要求这个递归函数中不要使用任何循环。然后在主程序中调用这个递归函数。

```
#include <iostream>
using namespace std;
int digitalRoot(int n){
    if(n<10) return n;
    else{
        int sum = 0;
        while(n>0){
            sum+= n%10;
            n/=10;
        }
        return digitalRoot(sum);
    }
}

int main(){
    int n;
    cout<<"please input the number:";
    cin>>n;
    cout<<"\nthe answer is:"<<digitalRoot(n)<<endl;
    return 0;
}
```

//或者：

```
int digitalRoot(int n) {
    if (n < 10) return n;
```

```

        else return digitalRoot(n % 10 + digitalRoot(n / 10));
    }

```

2. 任意进制转换问题。

【问题描述】编程输入任意十进制数 $N(N:-32767 \sim 32767)$ ，请输出它对应的二进制、八进制、十六进制；（注意，当数值超过十之后，就用字母 A、B、C.....来代替）

比如十六进制数：A1F,代表它最低位是 F,也就是 15，第二位是 1，最高位是 A，也就是 10，转换成 10 进制就是 $10 \times 16 \times 16 + 1 \times 16 + 15 = 2581$ 。

```

void printInt(int n,int base){
    if (n<base) cout.put(((n<10) ? n+'0' : n-10+'A'));
    else{
        printInt(n/base,base);
        n%=base;
        cout.put(((n<10) ? n+'0' : n-10+'A'));
    }
}

```

3. 递归

请用递归编写一个函数 `bool isPalindrome(int num)`，判断一个数字是否是回文（palindrome），所谓回文，就是从左边读与从右边读完全一样的数。例如：121 是一个回文，1221 也是一个回文。主程序首先要求用户输入任意不超过 10 位的数字，然后调用你编写的函数，判断用户的输入是否是回文。

```

int isPalindrome(int num,int reserved){
    if( num==0 )return reserved ;
    else{
        return isPalindrome(num/10,reserved*10+num%10);
    }
}

bool isPalindrome(int num){
    if(num<10) return true;
    return num==reverse(num);
}

int reverse(int num){

```

```

int rev=0;
while(num>0){
    rev=rev*10+num%10;
    num/=10;
}
return rev;
}

```

4. 打印杨辉三角形

由杨辉三角形可以看出每行的数存在如下规律：每行数据的个数和行序相同；每行的第一个数和最后一个数都是 1；中间的数为上一行中前一系列的数和后一系列的数之和。

						1						
					1		1					
				1		2		1				
			1		3		3		1			
		1		4		6		4		1		
	1		5		10		10		5		1	
1		6		15		20		15		6		1

要求输入行数 n ，输出打印 n 层的杨辉三角

```

#include <iostream>
using namespace std;
void print_line(int row_num, int column_num, int combination_num) {
    cout << combination_num << "\t\t"; //输出组合数
    combination_num *= (row_num - column_num);
    combination_num /= (column_num + 1);
    column_num++;
    if (column_num < row_num + 1) {
        print_line(row_num, column_num, combination_num);
    }
}
//n 阶三角，第 i 行需要先输出的空格数
void print_space(int n, int i) {
    for(int j = 0; j < n - i; j++) cout << "\t\t";
}

void print_tri(int n, int row_num) {
    for (row_num; row_num <= n; row_num++) {

```

```

        print_space(n, row_num);
        print_line(row_num, 0, 1); // the first num: Cn0=1
        cout << endl;

    }
}

int main() {
    // 杨辉三角形的阶数 n
    int n;
    cout << "please input the number of rows:";
    cin >> n;
    print_tri(n-1, 0);
    return 0;
}

```

5. 判断黑色星期五

问题描述：黑色星期五是指某天既是 13 号又是星期五。13 号在星期五的情况比在其他日子的少吗？为了回答这个问题，编写一个程序，计算每个月的 13 号分别落在周一到周日的次数。给出一个正整数 n （ n 不大于 400），要求计算从 1900 年 1 月 1 日至 1900+ n -1 年 12 月 31 日中 13 号落在周一到周日的次数。（已知 1900 年 1 月 1 日是星期一）

程序运行示例：

20

34 33 35 35 34 36 33

```

#include <iostream>
using namespace std;
// 判断是否为闰年
bool is_leap_year(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

int main() {
    int n;
    cin >> n;
    // 1900 年 1 月 1 日是星期一
    int day_of_week = 1; // 1 表示星期一，2 表示星期二，..., 7 表示星期日
    int days_in_month[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

```

```

int count[7] = {0}; // 记录 13 号落在周一到周日的次数
for (int year = 1900; year < 1900 + n; ++year) {
    days_in_month[1] = is_leap_year(year) ? 29 : 28; // 处理闰年 2 月的天数
    for (int month = 0; month < 12; ++month) {
        // 计算 13 号是星期几
        int day_13 = (day_of_week + 12) % 7;
        if (day_13 == 0) day_13 = 7; // 将 0 转换为 7（星期日）
        // 统计 13 号落在哪一天
        count[day_13 - 1]++;
        // 更新下个月 1 号的星期几
        day_of_week = (day_of_week + days_in_month[month]) % 7;
    }
}
// 输出结果
for (int i = 0; i < 7; ++i) {
    cout << count[i] << " ";
}
cout << endl;
return 0;
}

```

字符串

3. 多个单词转复数 【问题描述】编写一个函数 `void MultiplePluralForm(target[], source[])`; 将 `source[]` 字符串(长度不超过 200 个字符)中的单词依次转换为复数, 放入字符串数组 `target[]` 中, 要求调用第 2 题中的 `void RegularPluralForm(`

```

#include <iostream>
#include <cstring>
using namespace std;
void RegularPluralForm(char word[]);
bool isVowel(char ch);
void MultiplePluralForm(char target[], char source[]);
int main()
{
    char str1[201];
    char str2[400];
    cin.getline(str1, 201, '\n');

```

```

MultiplePluralForm(str2, str1);
cout << str2 << endl;
return 0;
}

void MultiplePluralForm(char target[], char source[]){
    char temp[201];
    int sourceLen = strlen(source);
    for(int i = 0; i < sourceLen; i++){
        if(temp[i] != ' ') temp[i] = source[i];
        else{
            char temp2[20];
            int lenth = strlen(temp);
            for(int j = 0; j < lenth; j++){
                temp2[j] = temp[j];
                if(j == strlen(temp)-1) temp2[j+1] = '\0';
            }
            RegularPluralForm(temp2);
            temp[0] = '\0';
            strcat(target, temp2);
            strcat(target, " ");
        }
    }
}

bool isVowel(char ch)
{
    switch(ch){
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u': return true;
        default: return false;
    }
}

void RegularPluralForm(char word[]){
    int len = strlen(word);

    if((word[len-1] == 's' || word[len-1] == 'x' || (word[len-2] == 'c' && word[len-1] == 'h') || (word[len-2]

```

```

== 's' && word[len-1] == 'h')) && strcmp(word, "quiz") != 0 && strcmp(word, "swiz") !=
0 && strcmp(word, "waltz") != 0){
    word[len] = 'e';
    word[len+1] = 's';
    word[len+2] = '\0';
}
else if (word[len-1] == 'y' && !isVowel(word[len-2])){
    word[len-1] = 'i';
    word[len] = 'e';
    word[len+1] = 's';
    word[len+2] = '\0';
}
else if (word[len-1] == 'f'){
    word[len-1] = 'v';
    word[len] = 'e';
    word[len+1] = 's';
    word[len+2] = '\0';
}
else if (word[len-2] == 'f' && word[len-1] == 'e'){
    word[len-2] = 'v';
    word[len-1] = 'e';
    word[len] = 's';
    word[len+1] = '\0';
}
else if (strcmp(word, "quiz") == 0 || strcmp(word, "swiz") == 0 || strcmp(word, "waltz")
== 0){
    word[len] = 'z';
    word[len+1] = 'e';
    word[len+2] = 's';
    word[len+3] = '\0';
}
else{
    word[len] = 's';
    word[len+1] = '\0';
}
}

```

4. 计算单词次数 **【问题描述】**编写一个程序，读入几行文本，并打印一个表格。此表格按照单词在文本中 出现的顺序，显示每个不同单词（不区分大小写）在文本中的出现次数。约定每个单词的长 度不超过 20 个字符，每行文本字符数不超过 80，输入文本中的单词个数不超过 1000 个。 **【提示及要求】** 1 可定义一个二维字符数组 `char words[1001][21]`，用于按照输入文本中单词出现的顺 序依次存储每个单词。2 编写一个函数 `ReadWords()` 从读入的几行文本中抽取每个单词，将其存储在二维数 组 `words` 中。 3 编写一个函数 `CountAndPrintWords()`，依次计算 `words` 数组中每个单词出现的次数， 并按照每行显示五个单词的格式，依次打印出每个单词出现的次数。 4 在 `main` 函数中，调用上述定义的函数。 程序运行实例： **【样例输入】** To be or not to be : that is the question, Whether it' s nobler in the mind to suffer **【样例输出】** to 3 be 2 or 1 not 1 that 1 is 1 the 2 question 1 whether 1 it' s 1 nobler 1 in 1 mind 1 suffer 1

```
#include<iostream>
#include<cstring>
#include <cctype>
#include <iomanip>
using namespace std;
int ReadWords(char words[][21]);
void CountAndPrintWords(char words[][21], int numOfWorks);
int main()
{
char words[1001][21];
int totalNumOfWorks;
totalNumOfWorks = ReadWords(words); // 从输入字符串中读取单词到字符串
数组 words 中

CountAndPrintWords(words, totalNumOfWorks); // 计算并输出 words 数组中
每个单词的重复次数

return 0;
}
int ReadWords(char words[][21]) {
char temp[1001];
cin.getline(temp, 1001);
char *token = strtok(temp, " "); //strtok 用法见 4_3
int count=0;
while(token!=NULL) {
```



```

        int i=0;
        char temp0[21];
        strcpy(temp0, token);
        int len=strlen(temp0);
        for(i=0;i<len;i++) temp0[i]=tolower(temp0[i]);
        strcpy(words[count], temp0);
        count++;
        token=strtok(NULL, " ");
    }
    return count;
}

void CountAndPrintWords(char words[][21], int numOfWords) {
    int wordCounts[1001]={0};
    bool printed[1001]={false};
    for(int i = 0;i<numOfWords;i++) {
        if(!printed[i]) { //判断是否与之前的重复（重复的再第一次出现的
遍历中都改为 true 了），若重复，则跳过
            int count = 1;
            for(int j = i+1;j<numOfWords;j++) {
                if(strcmp(words[i], words[j])==0) {
                    count++;
                    printed[j] = true; //把数组中其他与之重复的单词都置
为 true
                }
            }
            wordCounts[i] = count; //遍历一遍后，把每个单词的重复次数
记录下来
        }
    }

    int printedCount = 0;
    for(int k =0;k<numOfWords;k++) {
        if( ispunct(words[k][0])) printed[k] = true; //不打印标点
        if(!printed[k]) {
            cout<<words[k]<<"\t"<<wordCounts[k]<<"\t\t\t";
            printedCount++;
            if(printedCount%5==0) cout<<endl; //每五个换行
        }
    }
}

```

```
}
```

```
}
```

6. 字符串匹配 **【问题描述】**编写函数 `char* myStrstr(char* str1, char* str2)`, 找出 `str2` 字符串在 `str1` 字符串中第一次出现的位置(不包括 `str2` 的串结束符), 返回该位置的指针, 如找不到, 返回空指针。

```
char* myStrstr(char* str1, char* str2) {  
    int len1 = strlen(str1);  
    int len2 = strlen(str2);  
    if(len1 < len2) return NULL;  
    for(int i = 0; i <= len1 - len2; i++) {  
        for(int j = 0; j <= len2-1; j++) {  
            if(str1[i+j] != str2[j]) break;  
            if(j == len2-1) return str1+i;  
        }  
    }  
    return NULL;  
}
```

7. 获取指定字符串的子串 **【问题描述】**编写函数 `char* mysubstr(char* srcstr, int offset, int length)`, 该函数从 `srcstr` 中取得长度为 `length` 的子字符串。其中 `offset` 是起始字符的序号, `length` 是从 `offset` 开始所取的字符串的长度。注意, 此处要求所取得的子字符串占用新的内存空间(需要用 `new` 申请动态内存), 独立于原始字符串。主程序部分如下:

```
char* mysubstr(char* srcstr, int offset, int length) {  
    if (offset < 0 || length < 0) {  
        return NULL;  
    }  
    char *tmp = new char[length + 1];  
    for (int i = 0; i < length; i++) {  
        tmp[i] = srcstr[offset + i];  
    }  
    tmp[length] = '\0';  
    return tmp;  
}
```

类

题目四：

设计一个 `Image` 类，用于处理图像数据。类中有私有成员变量 `width`（图像宽度）、`height`（图像高度）和一个指向存储图像像素数据的二维指针 `pixels`（假设像素数据为整数类型，表示颜色值）。实现构造函数初始化图像宽度和高度并动态分配内存存储像素数据（初始化为 0），析构函数释放像素数据内存。编写成员函数 `setPixel` 用于设置指定位置的像素值，`getPixel` 用于获取指定位置的像素值，`printImage` 用于打印图像的像素数据（简单以矩阵形式输出）。在主函数中使用指针创建 `Image` 类对象，设置一些像素值并打印图像。

```
#include <iostream>
using namespace std;
class Image {
private:
    int width, height;
    int** pixels;
public:
    Image(int w, int h) : width(w), height(h) {
        pixels = new int*[height];
        for (int i = 0; i < height; ++i) {
            pixels[i] = new int[width];
            for (int j = 0; j < width; ++j) {
                pixels[i][j] = 0;
            }
        }
    }
    ~Image() {
        for (int i = 0; i < height; ++i) {
            delete[] pixels[i];
        }
        delete[] pixels;
    }

    void setPixel(int x, int y, int value) {
        if (x >= 0 && x < width && y >= 0 && y < height)
            pixels[y][x] = value;
    }
}
```

```

int getPixel(int x, int y) {
    if (x >= 0 && x < width && y >= 0 && y < height)
        return pixels[y][x];
}

void printImage() {
    for (int i = 0; i < height; ++i) {
        for (int j = 0; j < width; ++j) {
            cout << pixels[i][j] << "\t";
        }
        cout << endl;
    }
}

};

int main() {
    Image* img = new Image(5, 5);
    img->setPixel(0, 0, 255);
    img->setPixel(1, 1, 128);
    img->setPixel(2, 2, 64);
    img->setPixel(3, 3, 32);
    img->setPixel(4, 4, 16);
    img->printImage();
    delete img;
    return 0;
}

```

1、定义一个 `Date` 类，用于表示日期，包含年、月、日三个私有数据成员。重载加法运算符 `+`，实现日期加上一个整数（表示天数）后得到新的日期。重载减法运算符 `-`，实现两个日期相减，返回它们相差的天数（假设日期 1 大于日期 2）。考虑闰年和平年的情况以及月份天数的不同。

```

#include <iostream>
using namespace std;

```

```

class Date {

```

```

private:
    int year;
    int month;
    int day;
public:
// 构造函数，用于初始化日期对象。提供默认参数，默认日期为 2000 年 1 月 1 日
    Date(int y = 2000, int m = 1, int d = 1) : year(y), month(m), day(d)
    {}
// 重载加法运算符，实现日期加上指定天数
    Date operator+(int days) {
        Date temp(year, month, day);
        while(days>0) {
            int month_days[12]={31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
            if(temp.year%400||(temp.year%4==0&&temp.year%100!=0))
month_days[1]=29;
            if(temp.day+days<=month_days[temp.month-1]) {
                temp.day+=days;
                break;
            }
            else{
                days=days-month_days[temp.month-1]+temp.day;
                temp.day =0;
                if(temp.month==12) { temp.month=1; temp.year++;}
                else{temp.month++;}
            }
        }
        return temp;
    }
int howmanydays() const{
    int days=0;
    Date temp0(2020, 1, 1);
    for(;temp0.year<year;++temp0.year) {
        if(temp0.year%400||(temp0.year%4==0&&temp0.year%100!=0))
days+=366;
        else days+=365;
    }
}

```

```

        for(;temp0.month<month;++temp0.month) {
            int month_days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31};
            if (year % 400 || (year % 4 == 0 && year % 100 != 0)) month_days[1]
= 29;
            days+=month_days[temp0.month-1];
        }
        days+=day;
        return days;
    }
// 重载减法运算符，计算两个日期相差的天数
    int operator-(const Date& other) {return
howmanydays()-other.howmanydays();}
// 用于输出日期，格式为年-月-日
    void display() {
        cout << year << "-" << month << "-" << day << endl;
    }
};

int main() {
    Date d1(2024, 10, 15);
    Date d2(2024, 10, 5);
    Date d3 = d1 + 10;
    int diff = d1 - d2;
    d3.display();
    cout << "相差天数: " << diff << endl;
    return 0;
}

```